Regulatory Watch

Technical Architecture & State Report

Version 0.1.0 — March 2026

Engineering Team

March 27, 2026

Abstract

This document provides a comprehensive technical description of the **Regulatory Watch** platform, an AI-assisted regulatory monitoring system. It covers the application purpose and business rationale, the full distributed architecture, all five data ingestion connectors (Web, RSS, PDF, XML, and Email), the complete database schema, URL normalisation and spider-trap detection mechanisms, content deduplication strategy, S3 artifact storage design, current data collection status (188 documents collected as of March 2026), monitored regulatory sources, and an honest assessment of implemented versus planned capabilities. The document is intended for developers, architects, and technical stakeholders who need a single authoritative reference for the system.

Contents

1 Introduction and Purpose **4**
- 1.1 What is Regulatory Watch? . . . 4
- 1.2 Business Motivation . . . 4
- 1.3 Current Development Stage . . . 4

2 Technology Stack **4**

3 System Architecture **5**
- 3.1 Architectural Overview . . . 5
- 3.2 Service Map . . . 6
- 3.3 Docker Compose Services . . . 6
  - 3.3.1 Health Checks . . . 7
  - 3.3.2 Celery Worker and Beat Co-location . . . 7
- 3.4 FastAPI Application Structure . . . 7
- 3.5 Kafka Integration (Reserved) . . . 8

4 Database Schema **8**
4.1 Migration History . . . 8
4.2 Tables Overview . . . 8
4.3 The `raw_documents` Table . . . 8
4.3.1 Constraints and Indexes . . . 9
4.4 The `domains` Table . . . 10
4.5 The `urls` Table . . . 10
4.6 The `fetch_runs` and `fetch_attempts` Tables . . . 10

5 Data Ingestion Layer **10**
5.1 Design Principles . . . 10
5.2 Connector 1 — WebConnector (T2.2) . . . 11
5.2.1 Purpose . . . 11
5.2.2 Implementation . . . 11
5.2.3 Configuration Parameters . . . 11
5.2.4 Scheduled Runs (Celery Beat) . . . 11
5.3 Connector 2 — RSSConnector (T2.4) . . . 11
5.3.1 Purpose . . . 11
5.3.2 Implementation . . . 12
5.3.3 Scheduled Runs . . . 12
5.4 Connector 3 — PDFConnector (T2.3) . . . 12
5.4.1 Purpose . . . 12
5.4.2 Implementation . . . 12
5.4.3 Table Rendering . . . 12
5.4.4 Schedule . . . 13
5.5 Connector 4 — XMLConnector (T2.5) . . . 13
5.5.1 Purpose . . . 13
5.5.2 Supported XML Standards . . . 14
5.5.3 Implementation . . . 14
5.5.4 Schedule . . . 15
5.6 Connector 5 — EmailConnector (T2.6) . . . 15
5.6.1 Purpose . . . 15
5.6.2 Implementation . . . 15
5.6.3 Configuration . . . 15
5.6.4 Schedule . . . 16
5.7 Connector Schedule Summary . . . 16

6 Data Flow **16**
6.1 End-to-End Ingestion Pipeline . . . 16
6.2 Step-by-Step Description . . . 18

7 URL Normalisation and Spider-Trap Detection (T2.7) **18**
7.1 Overview . . . 18
7.2 URL Normalisation . . . 18
7.3 Domain Boundary Enforcement . . . 19
7.4 Spider-Trap Detection . . . 19

8 Deduplication Mechanism **20**
8.1 Design . . . 20

8.2 Three-Layer Deduplication . . . 20
8.3 Upsert Implementation . . . 20
8.4 Robustness Properties . . . 21

9 S3 Artifact Storage (T2.9) 21
9.1 Purpose . . . 21
9.2 Key Naming Convention . . . 21
9.3 Example S3 URIs . . . 21
9.4 Configuration . . . 21

10 Monitored Regulatory Sources 22
10.1 Configured Sources . . . 22
10.2 Financial Conduct Authority (FCA) . . . 22
10.3 EUR-Lex . . . 22
10.4 Federal Register / GovInfo (US) . . . 22

11 Current Data Collection Status 22
11.1 Documents Collected . . . 23
11.2 Analysis . . . 23

12 Application Configuration 23

13 Operational Procedures 25
13.1 Starting the Stack . . . 25
13.2 Running Migrations . . . 25
13.3 Checking Celery Tasks . . . 25
13.4 Triggering Manual Ingestion . . . 25
13.5 Heartbeat Monitoring . . . 25

14 What is Working and What is Not Yet Implemented 26
14.1 Implemented and Functional . . . 26
14.2 Not Yet Implemented . . . 26

15 Security Considerations 27

16 Project File Structure 27

17 Conclusion 28

A Celery Beat Schedule Reference 29

B Glossary 30

1 Introduction and Purpose

1.1 What is Regulatory Watch?

Regulatory Watch is a continuous regulatory intelligence platform designed to monitor, ingest, deduplicate, and eventually analyse regulatory publications from financial and legislative authorities around the world. The system automatically watches official government and regulatory body websites, RSS feeds, structured XML publications, PDF documents, and email alerts, collecting regulatory text as soon as it is published and making it available for downstream processing.

The platform targets compliance teams, legal analysts, and financial institutions that need timely, structured access to regulatory changes without manually visiting dozens of official websites every day.

1.2 Business Motivation

The volume of regulatory output from bodies such as the UK's Financial Conduct Authority (FCA), the European Union through EUR-Lex, and the United States Federal Register has grown substantially over the last decade. Missing a regulatory change can result in compliance failures, fines, and reputational damage. Manual monitoring is unreliable and expensive. Regulatory Watch addresses this by:

- Polling official feeds and websites on configurable schedules.
- Normalising and deduplicating content so analysts see each unique regulatory item exactly once.
- Storing raw artifacts for audit purposes.
- Providing a structured database that downstream AI analysis and alerting services can consume.

1.3 Current Development Stage

As of March 2026, the platform is in an active early-development phase. The data ingestion layer, deduplication engine, database schema, and background task infrastructure are fully operational. The platform has already collected **188 regulatory documents** from three jurisdictions. Higher-level capabilities—full-text search, AI-powered summarisation, change-diffing, and user-facing alerting—are planned for future sprints and are explicitly documented in Section 14.

2 Technology Stack

Table 1 lists every technology used in the platform.

Table 1: Technology Stack

Component	Version	Role
Python	3.11	Primary runtime language

Continued on next page...

Component	Version	Role
FastAPI	0.115.6	REST API framework; auto-generates OpenAPI docs
Uvicorn	0.34.0	ASGI server; serves the FastAPI application
SQLModel	0.0.22	ORM bridging SQLAlchemy + Pydantic; defines table models
PostgreSQL	16 (Alpine)	Primary relational store for documents and metadata
psycopg2-binary	2.9.10	PostgreSQL driver for SQLAlchemy
Alembic	1.14.1	Database schema migration manager
Celery	5.4.0	Distributed task queue; runs ingestion jobs
Redis	7 (Alpine)	Celery broker and result backend
Celery Beat	(built-in)	Periodic task scheduler embedded in the worker
Flower	2.0.1	Real-time Celery monitoring dashboard
Apache Kafka	7.6.0 (Confluent)	Event streaming bus (future AI pipeline integration)
Zookeeper	7.6.0 (Confluent)	Kafka coordination service
Crawl4AI	≥0.4.0	Playwright-based async web crawler; handles JS rendering
Playwright / Chromium	(via Crawl4AI)	Headless browser for JS-rendered regulatory pages
pdfplumber	≥0.11.0	Layout-aware PDF text and table extraction
feedparser	≥6.0.11	RSS 0.9–2.0, Atom, and RDF feed parsing
lxml	(transitive)	USLM and Akoma Ntoso XML parsing via XPath
httpx	0.28.1	Async HTTP client for robots.txt, PDF and XML fetching
langdetect	≥1.0.9	ISO 639-1 language detection (local, no API key)
boto3	≥1.34.0	AWS SDK; S3 artifact storage (optional)
kafka-python-ng	2.2.3	Kafka producer/consumer Python client
pydantic-settings	2.7.1	Typed environment-variable configuration
python-json-logger	3.2.1	Structured JSON log output
Docker / Compose	(latest)	Container orchestration for all services
OpenRouter / Qwen	API	LLM fallback for sparse web page extraction

3 System Architecture

3.1 Architectural Overview

Regulatory Watch follows a **microservice-inspired, event-driven architecture** deployed entirely in Docker containers via Docker Compose. The system decomposes into five logical tiers:

1. **API Tier** — FastAPI application serving REST endpoints.
2. **Worker Tier** — Celery worker executing ingestion tasks; Celery Beat schedules periodic polls.
3. **Data Tier** — PostgreSQL (structured storage) and Redis (broker/cache).
4. **Streaming Tier** — Apache Kafka with Zookeeper (event bus, reserved for AI pipeline).
5. **Artifact Tier** — AWS S3 (raw HTML and extracted text storage).

3.2 Service Map

Figure 1 shows the Docker Compose service topology.

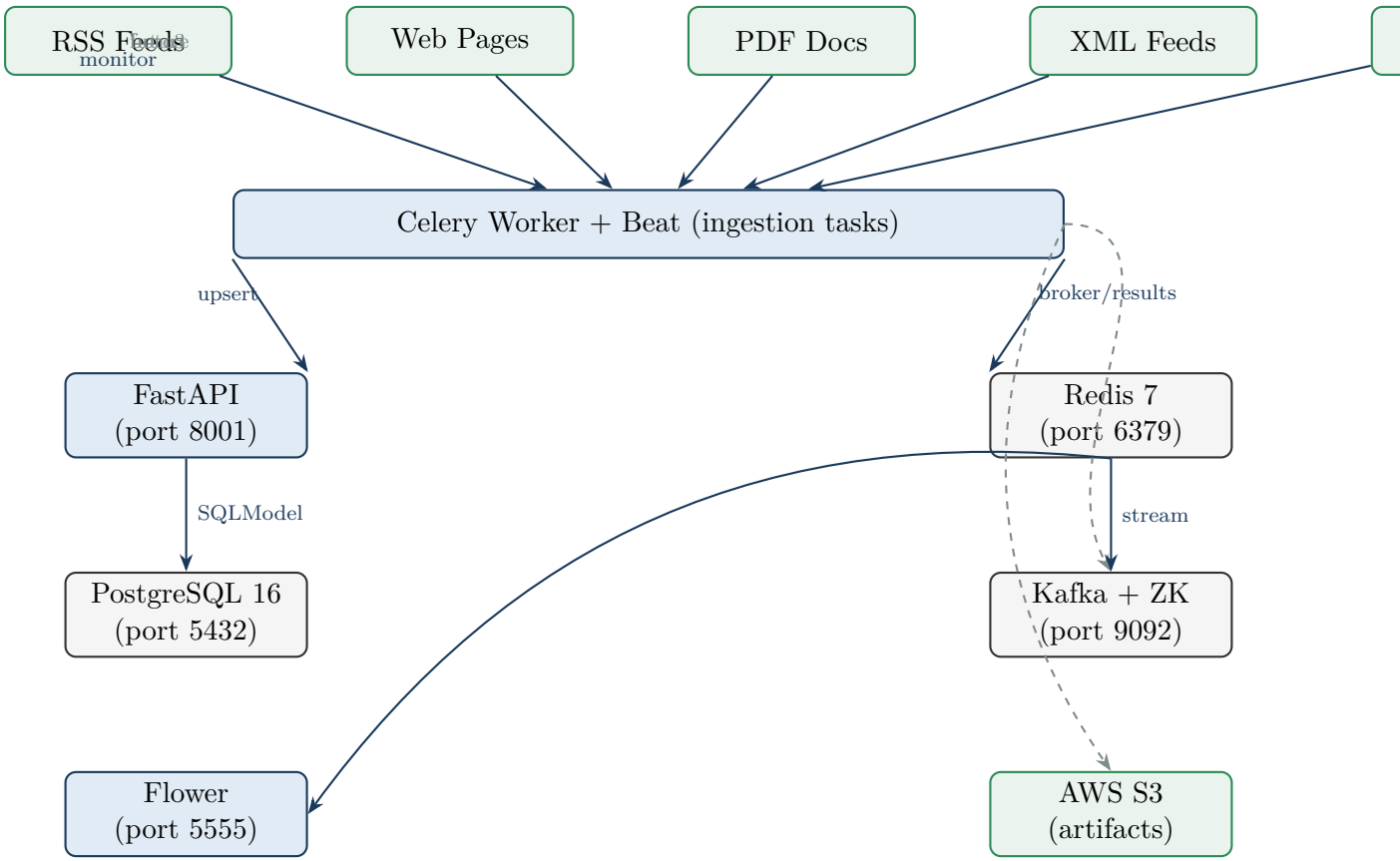

Figure 1: Regulatory Watch service topology

3.3 Docker Compose Services

The `docker-compose.yml` defines seven containers shown in Table 2.

Table 2: Docker Compose Services

Service	Image / Build	Host Port	Description
`db`	postgres:16-alpine	5432	PostgreSQL primary data store
`redis`	redis:7-alpine	6379	Celery broker and result backend
`zookeeper`	cp-zookeeper:7.6.0	2181	Kafka coordination
`kafka`	cp-kafka:7.6.0	9092	Event streaming (reserved)
`api`	./Dockerfile	8001	FastAPI application (port 8000 internal)
`worker`	./Dockerfile	—	Celery worker + Beat scheduler
`flower`	./Dockerfile	5555	Celery task monitoring UI

3.3.1 Health Checks

PostgreSQL and Redis both declare health-check commands in Compose. The `api` and `worker` services declare `depends_on` conditions that wait for healthy status before starting, preventing premature boot failures.

```
pg_isready -U regwatch
```

Listing 1: PostgreSQL health check

3.3.2 Celery Worker and Beat Co-location

The Celery worker and Beat scheduler run as a **single process** using the `-beat` flag:

```
celery -A app.celery_app:celery worker --loglevel=info --beat
```

Listing 2: Worker start command

This simplifies deployment for the current single-replica setup. In production, Beat would be separated into its own container to avoid Beat running on multiple replicas.

3.4 FastAPI Application Structure

The `api` service runs under Uvicorn with hot-reload enabled (`-reload`) and exposes the following routers:

Table 3: FastAPI Routers and Endpoints

Router	Prefix	Endpoints
health.py	`/health`	`GET /health`, `GET /health/db`, `GET /health/redis`
domains.py	`/domains`	`POST`, `GET` (list), `GET /{id}`, `PATCH /{id}`, `DELETE /{id}`
(root)	`/`	`GET /` — returns app name, version, and docs URL

Interactive API documentation is automatically available at `http://localhost:8001/docs` (Swagger UI) and `http://localhost:8001/redoc` (ReDoc).

3.5 Kafka Integration (Reserved)

Apache Kafka (Confluent Platform 7.6.0) with Zookeeper is fully provisioned and running in the Docker Compose stack. Auto-topic creation is enabled. The Kafka broker is available at `kafka:29092` (internal) and `localhost:9092` (host). At present, no Celery task publishes to Kafka. The infrastructure is reserved for a planned AI analysis pipeline that will consume raw documents from the `raw_documents` topic and publish structured analysis results.

4 Database Schema

4.1 Migration History

The schema is managed by Alembic with the following migration chain:

- **Migration 001** (`2026-03-24`): Initial schema — `domains`, `urls`, `fetch_runs`, `fetch_attempts`.
- **Migration 002** (`2026-03-24`): Add `raw_documents` table for the ingestion layer.

4.2 Tables Overview

The database contains five tables. Table 4 summarises their purpose and approximate row counts as of March 2026.

Table 4: Database Tables

Table	Purpose	Rows (approx.)
`domains`	Regulatory domains to monitor (seed URLs, rate limits)	small
`urls`	Individual pages within a domain; BFS frontier management	small
`fetch_runs`	Batch crawl execution records	small
`fetch_attempts`	Per-URL fetch records within a run; links to S3 artifacts	small
`raw_documents`	Ingested regulatory text from all connectors	**188**

4.3 The `raw_documents` Table

This is the primary output table of the ingestion layer. Every unique piece of regulatory content — regardless of source type — lands here.

Table 5: `raw_documents` Table Schema

Column	Type	Nullable	Description
`id`	UUID	NOT NULL	Primary key (UUID v4, auto-generated)
`source_url`	TEXT	NOT NULL	Canonical URL or identifier of the source
`source_type`	VARCHAR(20)	NOT NULL	One of: `web`, `pdf`, `rss`, `xml`, `email`
`raw_text`	TEXT	NULL	Full extracted body text
`title`	TEXT	NULL	Document title or section heading
`language`	VARCHAR(10)	NULL	ISO 639-1 language code (e.g. `en`, `fr`)
`content_hash`	VARCHAR(64)	NOT NULL	SHA-256 hex of `raw_text`; UNIQUE constraint
`fetched_at`	TIMESTAMPTZ	NOT NULL	When this document was first collected
`last_seen_at`	TIMESTAMPTZ	NOT NULL	When this content hash was last confirmed present

4.3.1 Constraints and Indexes

```sql
CREATE TABLE raw_documents (
    id             UUID          PRIMARY KEY,
    source_url     TEXT          NOT NULL,
    source_type    VARCHAR(20)   NOT NULL,
    raw_text       TEXT,
    title          TEXT,
    language       VARCHAR(10),
    content_hash   VARCHAR(64)   NOT NULL,
    fetched_at     TIMESTAMPTZ   NOT NULL,
    last_seen_at   TIMESTAMPTZ   NOT NULL,

    CONSTRAINT uq_raw_document_content_hash
        UNIQUE (content_hash),
    CONSTRAINT ck_raw_document_source_type
        CHECK (source_type IN ('web','pdf','rss','xml','email')
    )
);

CREATE INDEX ix_raw_documents_source_type  ON raw_documents (
    source_type);
CREATE INDEX ix_raw_documents_fetched_at   ON raw_documents (
    fetched_at);
CREATE INDEX ix_raw_documents_content_hash ON raw_documents (
    content_hash);
```

Listing 3: raw_documents DDL (from Alembic migration 002)

The `UNIQUE` constraint on `content_hash` is the enforcement anchor of the deduplication strategy (see Section 8). The three secondary indexes support efficient queries by source type, time range, and hash lookup.

4.4 The `domains` Table

Stores the set of domains under active monitoring. Each row carries a JSON array of seed URLs that the web crawler uses as starting points, plus a rate limit expressed in requests per second.

Key columns: `id` (UUID PK), `domain` (unique string, e.g. `fca.org.uk`), `seed_urls` (JSON array), `status` (`active` / `paused` / `archived`), `rate_limit_rps` (float).

4.5 The `urls` Table

Implements a per-domain URL frontier for the web crawler. Each discovered URL progresses through a state machine: `discovered` → `queued` → `fetched` (or `failed` / `ignored` / `blocked`). Key scoring columns include `priority`, `relevance_score`, `hub_score`, and `trap_score`. Scheduling is managed via `next_fetch_at` and `fetch_interval_hours` (default: 168 hours = weekly).

4.6 The `fetch_runs` and `fetch_attempts` Tables

`fetch_runs` records each batch crawl execution per domain (start time, finish time, counts of planned/fetched/changed/alerted URLs, and status). `fetch_attempts` records the result of fetching a single URL within a run: HTTP status code, content hash, diff status, and S3 URIs for the stored raw HTML and extracted text.

5 Data Ingestion Layer

5.1 Design Principles

The ingestion layer is built around a single abstract base class, `IngestorBase` (task T2.1), located in `app/ingestion/base.py`. All five connectors inherit from it and implement one async method:

```python
class IngestorBase(ABC):
    @abstractmethod
    async def fetch(self) -> List[RawDocument]:
        """
        Pull content from the source.
        content_hash (SHA-256 of raw_text) must be populated
        before returning so the storage layer can deduplicate.
        """
        ...
```

Listing 4: IngestorBase interface

The caller (a Celery task) invokes `connector.fetch()`, receives a list of `RawDocument` instances, and hands them to `upsert_documents()` in `app/ingestion/storage.py`. The connector is entirely unaware of persistence details; it only extracts and normalises.

5.2 Connector 1 — WebConnector (T2.2)

5.2.1 Purpose

The WebConnector performs a **breadth-first crawl** of a regulatory website, handling JavaScript-rendered pages, respecting robots.txt, enforcing rate limits, and applying spider-trap protection. It is the most complex of the five connectors.

5.2.2 Implementation

- **Library:** Crawl4AI with Playwright Chromium backend.
- **Content extraction:** Crawl4AI's Readability-style markdown processor strips navigation, footers, and ads. If the result is fewer than 200 characters, the system falls back to an LLM (Qwen via OpenRouter) that extracts the main regulatory body text.
- **Link discovery:** Internal links from the crawled page are normalised and enqueued for BFS traversal up to `max_depth` (default: 3).
- **Compliance:** robots.txt is fetched and cached once per host per crawl run. URLs blocked by robots.txt are skipped. A configurable `rate_limit_rps` parameter inserts `asyncio.sleep` between requests.

5.2.3 Configuration Parameters

Table 6: WebConnector Parameters

Parameter	Type	Default	Description
`seed_urls`	list[str]	—	Starting URLs for BFS
`allowed_domain`	str	—	Domain boundary (subdomains allowed)
`max_pages`	int	50	Hard cap on pages per run
`max_depth`	int	3	BFS depth limit from seeds
`rate_limit_rps`	float	1.0	Maximum requests per second

5.2.4 Scheduled Runs (Celery Beat)

Table 7: Scheduled Web Crawl Jobs

Domain	Schedule	Seed URL
EUR-Lex	Every 6 hours	`https://eur-lex.europa.eu/latest-laws/`
FCA	Every 6 hours	`https://www.fca.org.uk/news`
Federal Register	Every 6 hours	`https://www.federalregister.gov/documents/current`

5.3 Connector 2 — RSSConnector (T2.4)

5.3.1 Purpose

The RSSConnector polls RSS 0.9–2.0, Atom, and RDF feeds and turns each entry into a `RawDocument` with `source_type="rss"`.

5.3.2 Implementation

- **Library:** `feedparser` (supports all major feed formats, handles malformed feeds gracefully via `bozo` flag).
- **Since feedparser is synchronous**, it runs in a thread pool via `asyncio.run_in_executor`.
- **Text assembly:** Title, publication date (ISO 8601), summary, and link are joined into a single `raw_text` string. This provides consistent, searchable content per entry.
- **Date parsing:** Attempts `published_parsed` and `updated_parsed` (time structs), then falls back to raw RFC 2822 string parsing.
- **Language detection:** `langdetect` analyses up to the first 2000 characters.
- **In-run deduplication:** A `seen_hashes` set prevents duplicate entries within a single poll cycle before the data even reaches the database.
- **Max entries:** Capped at 100 entries per poll to avoid overwhelming the database on first run of a high-volume feed.

5.3.3 Scheduled Runs

Table 8: Scheduled RSS Jobs

Feed Name	Schedule	Feed URL
Federal Register	Every 30 minutes	`https://www.govinfo.gov/rss/fr.xml`
FCA News	Every 30 minutes	`https://www.fca.org.uk/news/rss.xml`

5.4 Connector 3 — PDFConnector (T2.3)

5.4.1 Purpose

The PDFConnector fetches a PDF document from an HTTP(S) URL or local file path and extracts text and tables, producing **one `RawDocument` per page**.

5.4.2 Implementation

- **Primary extractor:** IBM Docling, when installed. Docling provides layout-aware extraction with superior table structure understanding. The entire document is returned as a single markdown-formatted chunk.
- **Fallback extractor:** `pdfplumber`, which processes the PDF page by page. Non-table text is extracted with `page.extract_text()`, and tables are extracted with `page.extract_tables()` then rendered as comma-separated rows — preventing cells from being merged into a single unreadable string.
- **Page filtering:** Pages with fewer than 50 characters of extracted text are discarded.
- **Download:** httpx with a 60-second timeout, following redirects, using the `RegulatoryWatch/1.0` user agent.

5.4.3 Table Rendering

```python
def _table_to_text(table: list) -> str:
    lines = []
    for row in table:
        if row:
```

```
            cells = [str(c).strip() for c in row
                     if c is not None and str(c).strip()]
            if cells:
                lines.append(", ".join(cells))
    return "\n".join(lines)
```

Listing 5: Table-to-text rendering (pdfplumber)

Each table in the extracted text is surrounded by `-- TABLE --` / `-- END TABLE --` markers, making it easy for downstream parsers to detect tabular regions.

5.4.4 Schedule

PDF ingestion is triggered on-demand via the `pdf_ingest_task` Celery task. No periodic Beat schedule is currently defined for PDF sources; PDFs are submitted manually or triggered via API calls.

5.5 Connector 4 — XMLConnector (T2.5)

5.5.1 Purpose

The XMLConnector parses structured regulatory XML and produces one `RawDocument` per logical section or article.

5.5.2 Supported XML Standards

Table 9: XML Standards Supported

Standard	Namespace	Used by
USLM	`http://schemas.gpo.gov/xml/uslm`	US Code, Federal Register, GPO documents
Akoma Ntoso (AKN)	`http://docs.oasis-open.org/legaldocml/ns/akn/3.0`	UN, EU Parliament, many national legislatures
Generic	(fallback)	Any other XML; top-level child elements extracted

5.5.3 Implementation

- **Library:** `lxml` with XPath queries. Namespace detection is performed by inspecting the root element's `nsmap`.
- **USLM:** Extracts `<section>` nodes, collects headings (`<heading>`, `<num>`), body text, and effective date attributes (`effectiveDate`, `startDate`).
- **AKN:** Extracts `<section>`, `<article>`, and `<chapter>` nodes. Reads `<num>` and `<heading>` sub-elements and retrieves expression dates from `<FRBRdate>`.
- **Generic fallback:** Iterates direct children of the root element; any child with at least 50 characters of text becomes a section.

- **XPath fallback chain:** Each extractor tries multiple XPath expressions (namespaced first, then local-name based) to handle namespace declaration variations in real-world XML.

5.5.4 Schedule

XML ingestion is triggered on-demand via the `xml_ingest_task` Celery task.

5.6 Connector 5 — EmailConnector (T2.6)

5.6.1 Purpose

The EmailConnector reads unread messages from an IMAP mailbox and extracts the body text as `RawDocument` records with `source_type="email"`. PDF attachments found in emails are forwarded to the PDF extraction pipeline.

5.6.2 Implementation

- **Protocol:** IMAP with SSL (port 993 default) or STARTTLS. Uses Python's standard library `imaplib`.
- **Message selection:** Searches for `UNSEEN` (unread) messages only, capped at `max_emails` (default: 50) per run.
- **Body extraction:** For multipart messages, plain text parts are preferred over HTML. If only HTML is available, tags are stripped via regex. Headers (Subject, From, Date) are prepended to the body to provide context.
- **Header decoding:** RFC 2047 encoded-word headers are decoded via `email.header.decode_head`
- **PDF attachments:** Any `application/pdf` attachment is passed to the PDF extraction functions (`_extract_with_docling` / `_extract_with_pdfplumber`) and returns additional `RawDocument` records with `source_type="pdf"`.
- **Threading:** Since `imaplib` is synchronous, the entire IMAP session runs in `asyncio.run_in_executor` to avoid blocking the event loop.

5.6.3 Configuration

Table 10: EmailConnector Environment Variables

Variable	Default	Description
`IMAP_HOST`	(empty)	IMAP server hostname
`IMAP_PORT`	993	IMAP port
`IMAP_USER`	(empty)	Login email address
`IMAP_PASSWORD`	(empty)	Password or app-specific password
`IMAP_MAILBOX`	INBOX	Folder to monitor

If `IMAP_HOST` or `IMAP_USER` is not set, the `email_ingest_task` logs a skip message and returns immediately without connecting.

5.6.4 Schedule

The `email_ingest_task` Celery task is available and functional. Periodic scheduling via Beat is not yet configured by default but can be added to `celery_app.py`'s `beat_schedule` dictionary.

5.7 Connector Schedule Summary

Table 11: Ingestion Schedule Summary

Connector	Source Type	Schedule	Retry Policy
WebConnector (EUR-Lex)	web	Every 6 hours	3 retries, exponential backoff
WebConnector (FCA)	web	Every 6 hours	3 retries, exponential backoff
WebConnector (FederalRegister)	web	Every 6 hours	3 retries, exponential backoff
RSSConnector (govinfo.gov)	rss	Every 30 minutes	3 retries, exponential backoff
RSSConnector (fca.org.uk)	rss	Every 30 minutes	3 retries, exponential backoff
PDFConnector	pdf	On-demand	3 retries, exponential backoff
XMLConnector	xml	On-demand	3 retries, exponential backoff
EmailConnector	email	Not scheduled yet	3 retries, exponential backoff
Heartbeat	—	Every 5 minutes	—

6 Data Flow

6.1 End-to-End Ingestion Pipeline

Figure 2 describes the complete journey from a remote regulatory source to persistent storage.

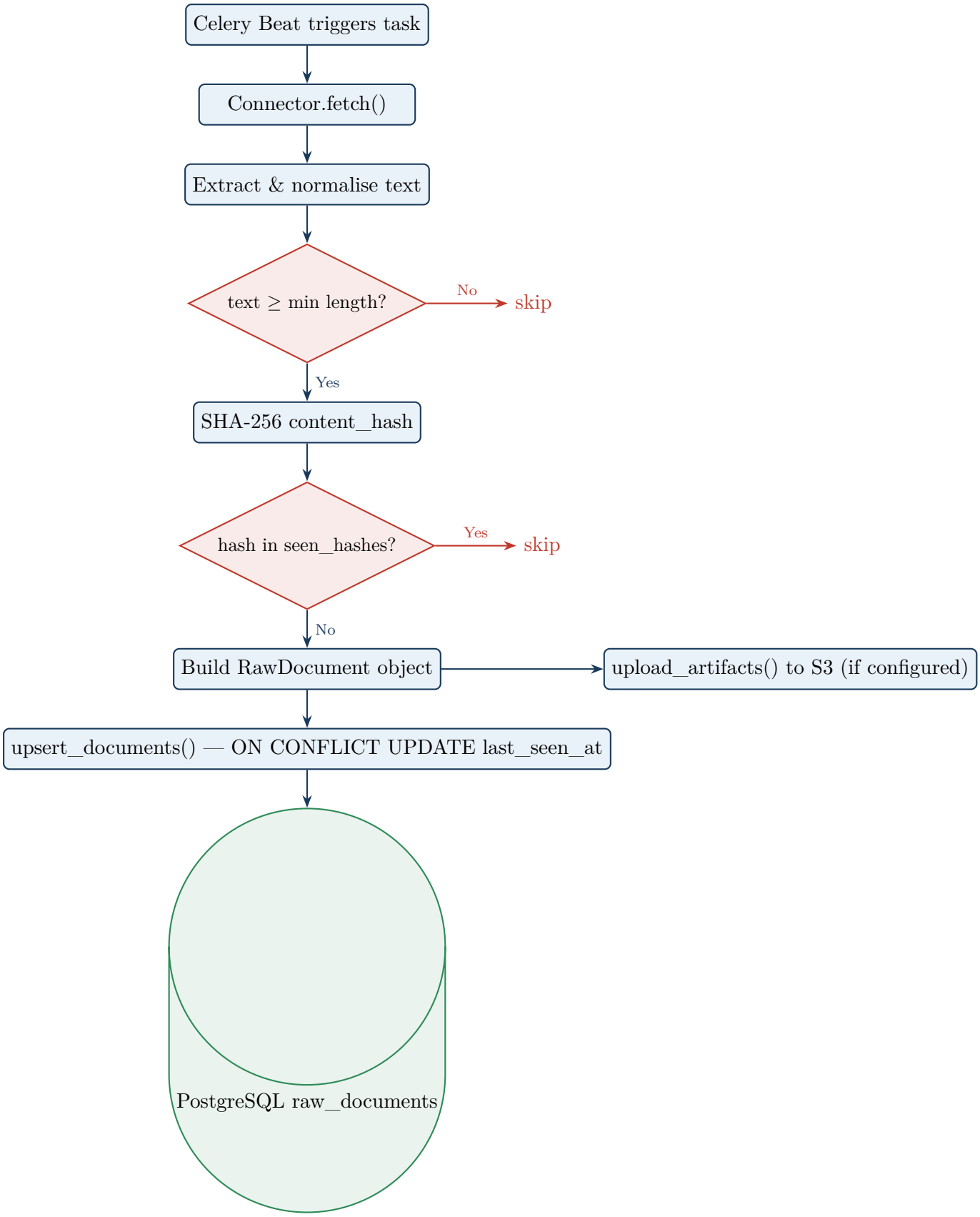

Figure 2: Data flow from source to storage

6.2 Step-by-Step Description

1. **Trigger:** Celery Beat fires a task (e.g. `rss_ingest_task`) on its configured schedule.
2. **Fetch:** The appropriate connector downloads content from the remote source. Web pages are rendered via Playwright. RSS feeds are parsed by feedparser. PDFs are downloaded via httpx and extracted by pdfplumber or Docling. XML is fetched and parsed by lxml.
3. **Extract & Normalise:** Raw text is extracted. For web pages, Readability-processed markdown is used; if too sparse, the LLM fallback runs. For RSS, title+summary+link are concatenated. For PDFs, text is extracted per page including tables. For XML, section text is gathered recursively with `itertext()`.
4. **Minimum-length filter:** Content shorter than a connector-specific threshold (20–200 characters depending on connector) is discarded.
5. **SHA-256 hash:** The normalised `raw_text` is hashed with SHA-256.
6. **In-run deduplication:** The hash is checked against a `seen_hashes` set maintained for the current run. Duplicates within the same fetch cycle are dropped before any database call.
7. **RawDocument construction:** A `RawDocument` SQLModel instance is built with all fields populated.
8. **Upsert:** `upsert_documents()` executes a PostgreSQL `INSERT ...ON CONFLICT DO UPDATE` statement. New documents are inserted; existing ones (same hash) have only `last_seen_at` updated.
9. **S3 upload (optional):** If AWS credentials and a bucket are configured, raw HTML and/or extracted text are uploaded to S3 under the key pattern `artifacts/{source_type}/{YYYY-MM-`

7 URL Normalisation and Spider-Trap Detection (T2.7)

7.1 Overview

URL normalisation and spider-trap detection are implemented in `app/ingestion/url_utils.py` and are called by the WebConnector before adding any discovered link to the BFS queue.

7.2 URL Normalisation

The `normalize_url()` function produces a stable, canonical URL form through the following transformation pipeline:

1. **Relative URL resolution:** If a `base_url` is provided, relative URLs are resolved against it using `urllib.parse.urljoin`.
2. **Scheme validation:** Only `http` and `https` schemes are accepted; everything else (`ftp://`, `mailto:`, `javascript:`, etc.) returns `None`.
3. **Tracking parameter removal:** 18 known tracking parameters are stripped from the query string.
4. **Parameter sorting:** Remaining query parameters are sorted alphabetically for a consistent canonical form (e.g. `?b=2&a=1` → `?a=1&b=2`).
5. **Fragment removal:** The `#fragment` component is always removed.
6. **Case normalisation:** Scheme and host are lowercased.
7. **Trailing slash removal:** Trailing slashes are removed from the path component (except the root `/`).

The full set of stripped tracking parameters includes:

```
_TRACKING_PARAMS: frozenset[str] = frozenset({
    "utm_source", "utm_medium", "utm_campaign", "utm_term",
    "utm_content", "utm_id", "utm_referrer",
    "fbclid", "gclid", "msclkid", "ttclid",
    "_ga", "_gl", "ref", "referrer", "mc_cid", "mc_eid",
})
```

Listing 6: Tracking parameters stripped during normalisation

7.3 Domain Boundary Enforcement

`is_same_domain(url, allowed_domain)` returns `True` only if the URL's host is exactly `allowed_domain` or a subdomain of it. Both sides are stripped of a leading `www.` for stable comparison, preventing false positives.

```
is_same_domain("https://www.cbp.gov/trade/rules", "cbp.gov")  # True
is_same_domain("https://evil.com/cbp.gov", "cbp.gov")  # False
is_same_domain("https://api.fca.org.uk/data", "fca.org.uk")  # True
```

Listing 7: Domain boundary check examples

7.4 Spider-Trap Detection

`is_spider_trap(url)` returns `True` if any of four trap signatures are detected:

Table 12: Spider-Trap Detection Rules

Rule	Trigger Condition	Example
URL length	URL longer than 512 characters	Long concatenated parameter strings
Session ID	Query string contains `session_id`, `sid`, `jsessionid`, `phpsessid`, `aspsessionid`	`/page?sessionid=abc123`
Calendar loop	Path contains a date segment `/YYYY/MM(/DD)?/`	`/news/2024/01/15/`
Repeated path	Same path segment repeated 3+ times	`/a/b/a/b/a/b/`

Calendar-loop detection uses the regex `/\d{4}/(?:0[1-9]|1[0-2])(?:/...)?(?:/|$)` which matches year/month and optional day segments. Repeated-segment detection uses `(/[^/?#]{2,})\1{2,}` which catches any path segment repeated two or more additional times.

8 Deduplication Mechanism

8.1 Design

Every `RawDocument` carries a **SHA-256 hex digest** of its `raw_text` content in the `content_hash` column. This 64-character string is the deduplication key at every level of the system.

```python
def _sha256(text: str) -> str:
    return hashlib.sha256(
        text.encode("utf-8", errors="replace")
    ).hexdigest()
```

Listing 8: SHA-256 content hash computation (shared across all connectors)

8.2 Three-Layer Deduplication

1. **Within-run deduplication:** Each connector maintains a `seen_hashes: set[str]` in memory during a single `fetch()` call. Any document whose hash is already in the set is skipped immediately, preventing redundant DB calls.
2. **Database-level deduplication:** The `raw_documents` table has a `UNIQUE` constraint on `content_hash`. The storage layer uses a PostgreSQL `INSERT ...ON CONFLICT DO UPDATE` (upsert) statement that, on collision, only updates `last_seen_at`. This means a document seen again does not produce a duplicate row but its last-seen timestamp is refreshed.
3. **Cross-run identity:** Because the hash is derived from the full text content and not the URL, two different URLs with identical content will correctly deduplicate to a single row. Conversely, the same URL with changed content will produce a new row.

8.3 Upsert Implementation

```python
stmt = (
    pg_insert(RawDocument)
    .values(
        id=doc.id,
        source_url=doc.source_url,
        source_type=doc.source_type,
        raw_text=doc.raw_text,
        title=doc.title,
        language=doc.language,
        content_hash=doc.content_hash,
        fetched_at=doc.fetched_at,
        last_seen_at=now,
    )
    .on_conflict_do_update(
        constraint="uq_raw_document_content_hash",
        set_={"last_seen_at": now},
    )
)
```

Listing 9: PostgreSQL upsert in storage.py

8.4 Robustness Properties

- **Idempotent:** Re-running any ingestion task for the same unchanged source produces no new database rows.
- **Race-condition safe:** The PostgreSQL `ON CONFLICT` clause is atomic; concurrent Celery worker processes cannot produce duplicate rows from simultaneous inserts of the same hash.
- **Content-addressed:** Minor formatting changes that alter the text (e.g. a regulatory body updates its website navigation structure) will produce a new hash and a new row. The `last_seen_at` column tracks when a hash was last observed, enabling future change-detection logic.

9 S3 Artifact Storage (T2.9)

9.1 Purpose

S3 artifact storage preserves the raw HTML and extracted text of fetched documents for audit, replay, and debugging. It is entirely optional; if AWS credentials are not configured, the upload is silently skipped.

9.2 Key Naming Convention

`artifacts/{source_type}/{YYYY-MM-DD}/{content_hash[:8]}_raw.html`
`artifacts/{source_type}/{YYYY-MM-DD}/{content_hash[:8]}_text.txt`

The first 8 hex characters of the content hash act as a short identifier. The date prefix organises artifacts chronologically and enables S3 lifecycle policies (e.g. transition to Glacier after 90 days).

9.3 Example S3 URIs

```
s3://regwatch-artifacts/artifacts/web/2026-03-27/a1b2c3d4_raw.
    html
s3://regwatch-artifacts/artifacts/rss/2026-03-27/e5f6a7b8_text.
    txt
s3://regwatch-artifacts/artifacts/pdf/2026-03-27/c9d0e1f2_text.
    txt
```

9.4 Configuration

Table 13: S3 Configuration Environment Variables

Variable	Default	Description
`AWS_S3_BUCKET`	(empty)	Target S3 bucket name; if empty, uploads are skipped
`AWS_REGION`	us-east-1	AWS region
`AWS_ACCESS_KEY_ID`	(empty)	AWS access key
`AWS_SECRET_ACCESS_KEY`	(empty)	AWS secret key

If boto3 is not installed or credentials are missing, the `_get_s3_client()` function returns `(None, None)` and the upload function returns an empty dict without raising an exception.

10 Monitored Regulatory Sources

10.1 Configured Sources

Table 14 lists all regulatory sources currently configured in the Celery Beat schedule.

Table 14: Active Regulatory Sources

Authority	Jurisdiction	Connector	Schedule	URL
FCA	UK	RSS	30 min	`fca.org.uk/news/rss.xml`
FCA	UK	Web	6 h	`www.fca.org.uk/news`
EUR-Lex	EU	Web	6 h	`eur-lex.europa.eu/latest-laws/`
GovInfo	US	RSS	30 min	`govinfo.gov/rss/fr.xml`
Federal Register	US	Web	6 h	`federalregister.gov/documents/curre`

10.2 Financial Conduct Authority (FCA)

The FCA is the UK's primary financial services regulator. Its news RSS feed (`fca.org.uk/news/rss.xml`) provides policy statements, guidance consultations, enforcement notices, and market watch updates. The web crawler targets the FCA news index page and discovers linked articles. The FCA source accounts for the majority of the RSS data collected.

10.3 EUR-Lex

EUR-Lex is the official portal for European Union law, hosting the Official Journal of the EU, EU regulations, directives, decisions, and legal acts. The web crawler seeds from `eur-lex.europa.eu/latest-laws/` at a conservative rate of 0.5 RPS (one request every 2 seconds) to respect server capacity. EUR-Lex documents are often available as both HTML and structured PDF, with some in Akoma Ntoso XML format (handled by the XMLConnector).

10.4 Federal Register / GovInfo (US)

The Federal Register is the official daily journal of the US federal government, publishing proposed and final rules, executive orders, and notices from all federal agencies. Two collection paths exist: the RSS feed at `govinfo.gov/rss/fr.xml` provides a daily summary, and the web crawler targets `federalregister.gov/documents/current` for full document text. GovInfo also publishes USLM-formatted XML (handled by the XMLConnector).

11 Current Data Collection Status

11.1 Documents Collected

As of March 2026, the `raw_documents` table contains **188 documents**. Table 15 provides a breakdown by source type.

Table 15: Raw Documents by Source Type (March 2026)

Source Type	Count	Percentage
RSS	122	64.9%
Web	60	31.9%
XML	4	2.1%
PDF	2	1.1%
Email	0	0.0%
Total	**188**	**100%**

11.2 Analysis

- **RSS dominates** because both FCA and GovInfo RSS feeds are polled every 30 minutes and each provides up to 100 entries per cycle. RSS is the highest-velocity source.
- **Web crawler documents** (60) come from the EUR-Lex, FCA, and Federal Register web crawls. At 50 pages per 6-hour run, growth from the web connector is steady but more conservative.
- **XML documents** (4) represent a small number of manually triggered or test runs of the XMLConnector against USLM-formatted Federal Register XML.
- **PDF documents** (2) represent manually triggered PDF test runs.
- **Email documents** (0) is expected: the EmailConnector requires IMAP credentials which are not configured in the current deployment.

Ingestion Status

All automated connectors (RSS, Web) are operational and collecting data continuously. The deduplication mechanism is working correctly: repeated runs do not create duplicate rows. The 188 documents represent 188 *unique* content hashes.

12 Application Configuration

All configuration is managed via environment variables, loaded by Pydantic Settings. A `.env` file can override defaults for local development.

Table 16: Application Environment Variables

Variable	Default	Description
`APP_NAME`	Regulatory Watch	Application name
`APP_VERSION`	0.1.0	API version string
`DEBUG`	false	Enable SQLAlchemy query logging
`DATABASE_URL`	postgresql://regwatch:. . . @db:5432/regwatch	PostgreSQL DSN
`REDIS_URL`	redis://redis:6379/0	Redis connection URL
`KAFKA_BOOTSTRAP_SERVERS`	kafka:29092	Kafka broker addresses
`OPENROUTER_API_KEY`	(empty)	OpenRouter API key for LLM fallback
`OPENROUTER_MODEL`	qwen/qwen3-next-80b-. . .	Model identifier
`OPENROUTER_BASE_URL`	https://openrouter.ai/api/v1	OpenRouter base URL
`AWS_S3_BUCKET`	(empty)	S3 bucket for artifacts
`AWS_REGION`	us-east-1	AWS region
`AWS_ACCESS_KEY_ID`	(empty)	AWS credentials
`AWS_SECRET_ACCESS_KEY`	(empty)	AWS credentials
`IMAP_HOST`	(empty)	IMAP server hostname
`IMAP_PORT`	993	IMAP port
`IMAP_USER`	(empty)	IMAP login user
`IMAP_PASSWORD`	(empty)	IMAP password
`IMAP_MAILBOX`	INBOX	IMAP folder

13 Operational Procedures

13.1 Starting the Stack

```
cd regulation-prj-v1
docker compose up --build -d
```

Listing 10: Start all services

13.2 Running Migrations

```
docker compose exec api alembic upgrade head
```

Listing 11: Apply Alembic migrations

13.3 Checking Celery Tasks

Flower is available at `http://localhost:5555`. It provides real-time monitoring of:

- Active, scheduled, and completed tasks.
- Worker status and concurrency.
- Task failure rates and retry counts.
- Per-task execution time statistics.

13.4 Triggering Manual Ingestion

Individual ingestion tasks can be triggered manually from a Python shell or via Celery's API:

```
from app.celery_app import rss_ingest_task
result = rss_ingest_task.delay(
    feed_url="https://www.fca.org.uk/news/rss.xml"
)
print(result.get(timeout=60))
# {'feed_url': '...', 'fetched': 50, 'inserted': 12, 'updated':
    38}
```

Listing 12: Trigger a manual RSS poll

```
from app.celery_app import pdf_ingest_task
pdf_ingest_task.delay(
    source="https://www.fca.org.uk/publication/policy/ps21-3.
   pdf",
    title="FCA Policy Statement PS21/3"
)
```

Listing 13: Trigger a manual PDF extraction

13.5 Heartbeat Monitoring

A dedicated heartbeat task runs every 5 minutes, pings both Redis and PostgreSQL, and logs a structured JSON status line. The log can be monitored to detect infrastructure-level failures.

14 What is Working and What is Not Yet Implemented

14.1 Implemented and Functional

Fully Operational

- Full Docker Compose stack: PostgreSQL, Redis, Zookeeper, Kafka, FastAPI, Celery Worker + Beat, Flower.
- All 5 ingestion connectors: Web (Crawl4AI + Playwright), RSS (feedparser), PDF (pdfplumber + Docling), XML (lxml / USLM / AKN), Email (IMAP).
- SHA-256 content deduplication with PostgreSQL upsert (three-layer).
- URL normalisation: tracking parameter removal, fragment stripping, parameter sorting, case normalisation.
- Spider-trap detection: URL length cap, session ID detection, calendar loop detection, repeated path segment detection.
- Domain boundary enforcement in the web crawler.
- robots.txt compliance in the web crawler.
- Configurable rate limiting (per domain, per connector).
- Language detection (ISO 639-1) via langdetect.
- S3 artifact storage (boto3, optional).
- Alembic database migrations (2 migrations applied).
- Structured JSON logging via python-json-logger.
- Health-check REST endpoints (`/health`, `/health/db`, `/health/redis`).
- Domain CRUD REST API (`/domains`).
- Celery task retry with exponential backoff (up to 3 retries).
- 188 real regulatory documents already collected.
- LLM fallback (OpenRouter/Qwen) for sparse web pages (requires API key).

14.2 Not Yet Implemented

Planned Features — Not Yet Implemented

1. **Full-text search:** No search endpoint exists. The `raw_documents` table lacks a full-text search index (`tsvector/GIN`). Planned: `GET /documents?q={query}` endpoint backed by PostgreSQL FTS or Elasticsearch.
2. **AI analysis pipeline:** No LLM summarisation, topic extraction, or regulatory impact analysis has been built. The Kafka bus is provisioned and waiting. Planned: a Kafka consumer reads from the `raw_documents` topic, calls an LLM, and writes structured analysis to an `analysed_documents` table.
3. **Change detection / diff engine:** No mechanism currently identifies when a previously seen document has changed in content. The `last_content_hash` on the `urls` table supports this, but the diff logic is not implemented.
4. **Alerting and notifications:** No email, Slack, or webhook alerts are sent when new regulatory content is detected. Planned: a notification service that watches the Kafka topic and dispatches alerts.

5. **User-facing frontend/UI:** No web interface exists. The FastAPI Swagger UI (`/docs`) is the only browser interface.
6. **Document REST API:** No `/documents` endpoint exists for querying collected documents.
7. **Relevance scoring:** The `relevance_score` and `hub_score` columns on `urls` are populated with defaults but not yet computed by any algorithm.
8. **Email connector scheduling:** The EmailConnector is functional but no Beat periodic schedule is configured.
9. **Authentication / authorisation:** The FastAPI application has no auth. All endpoints are publicly accessible. OAuth2/JWT is planned.
10. **Metrics and observability:** No Prometheus metrics endpoint. No distributed tracing (OpenTelemetry). Only structured JSON logs are currently available.
11. **Multi-replica worker scaling:** The Beat scheduler is embedded in the worker process, which prevents horizontal scaling. Separation of Beat into a standalone service is needed for production.
12. **RSS pagination:** The RSS connector is capped at 100 entries per poll. Historical backfill of older feed entries is not implemented.

15 Security Considerations

- **Credentials:** All secrets (database passwords, AWS keys, IMAP password, OpenRouter API key) are passed as environment variables, not hardcoded. Production deployments should use Docker secrets or a vault solution.
- **robots.txt:** The web crawler respects robots.txt. Non-200 responses are treated as "allow all" to handle WAF-blocked robots.txt files, which is a pragmatic compromise. A stricter policy may be appropriate for some jurisdictions.
- **Rate limiting:** Per-domain rate limits are configurable (default: 1 RPS; crawl tasks use 0.5 RPS for regulatory sites). This reduces load on target servers and lessens the risk of IP blocking.
- **User agent:** A transparent user agent string is used: `RegulatoryWatch/1.0 (compliance monitoring bot; +https://github.com/your-org/regulatory-watch)`.
- **CORS:** The API currently allows all origins (`allow_origins=["*"]`). This must be restricted for production deployment.
- **No authentication:** All API endpoints are unauthenticated (see Section 14).

16 Project File Structure

```
regulation-prj-v1/
|-- app/
|   |-- celery_app.py          # Celery instance, Beat schedule, all tasks
|   |-- config.py              # Pydantic Settings (env vars)
|   |-- database.py            # SQLAlchemy engine, session factory
|   |-- main.py                # FastAPI app, lifespan, middleware, routers
```

```
|    |-- models.py              # SQLModel table definitions (5 models)
|    |-- schemas.py             # Pydantic API schemas
|    |-- ingestion/
|    |    |-- base.py           # IngestorBase abstract class
|    |    |-- artifact_store.py # S3 upload helper
|    |    |-- email_connector.py
|    |    |-- pdf_connector.py
|    |    |-- rss_connector.py
|    |    |-- storage.py        # upsert_documents()
|    |    |-- url_utils.py      # normalize_url, is_spider_trap, etc.
|    |    |-- web_connector.py
|    |    `-- xml_connector.py
|    `-- routers/
|         |-- domains.py        # CRUD for /domains
|         `-- health.py         # /health, /health/db, /health/redis
|-- alembic/
|    |-- versions/
|    |    |-- 001_initial_schema.py
|    |    `-- 002_add_raw_documents.py
|    `-- env.py
|-- docs/
|    `-- regulatory_watch_report.tex   (this document)
|-- alembic.ini
|-- docker-compose.yml
|-- Dockerfile
|-- Makefile
|-- requirements.txt
`-- README.md
```

Listing 14: Repository layout

17 Conclusion

The Regulatory Watch platform has a solid, production-grade foundation. The ingestion layer is fully operational with five distinct connector types, robust deduplication, URL canonicalisation, spider-trap protection, and S3 artifact persistence. The distributed task infrastructure (Celery + Redis + Beat) runs reliably and has already accumulated 188 unique regulatory documents from the FCA, EUR-Lex, and the US Federal Register.

The system is designed for extension. Adding a new regulatory source requires only configuring a new Beat schedule entry and providing seed URLs or a feed URL. The connector abstraction (`IngestorBase`) makes it straightforward to add new source types (e.g. a GraphQL connector, a scraper for a specific agency's API).

The next major milestones are:

1. Implementing the document retrieval REST API (`/documents`) with full-text search.
2. Building the Kafka-based AI analysis pipeline for summarisation and topic tagging.

3. Implementing change detection and regulatory alerting.
4. Adding authentication and hardening the API for external exposure.

The codebase follows Python best practices throughout: type annotations, async-first design, structured logging, environment-variable configuration, Alembic-managed migrations, and separation of concerns between the connector, storage, and API layers.

A Celery Beat Schedule Reference

```
celery.conf.beat_schedule = {
    "heartbeat-every-5minutes": {
        "task": "app.celery_app.heartbeat",
        "schedule": 300.0,          # 5 minutes
    },
    "rss-federal-register-every-30min": {
        "task": "app.celery_app.rss_ingest_task",
        "schedule": 1800.0,         # 30 minutes
        "kwargs": {
            "feed_url": "https://www.govinfo.gov/rss/fr.xml",
        },
    },
    "rss-fca-every-30min": {
        "task": "app.celery_app.rss_ingest_task",
        "schedule": 1800.0,
        "kwargs": {
            "feed_url": "https://www.fca.org.uk/news/rss.xml",
        },
    },
    "web-crawl-eur-lex-every-6h": {
        "task": "app.celery_app.web_crawl_task",
        "schedule": 21600.0,        # 6 hours
        "kwargs": {
            "seed_urls": ["https://eur-lex.europa.eu/latest-
    laws/"],
            "allowed_domain": "eur-lex.europa.eu",
            "max_pages": 50,
            "rate_limit_rps": 0.5,
        },
    },
    "web-crawl-fca-every-6h": {
        "task": "app.celery_app.web_crawl_task",
        "schedule": 21600.0,
        "kwargs": {
            "seed_urls": ["https://www.fca.org.uk/news"],
            "allowed_domain": "www.fca.org.uk",
            "max_pages": 50,
            "rate_limit_rps": 0.5,
        },
    },
    "web-crawl-federal-register-every-6h": {
        "task": "app.celery_app.web_crawl_task",
```

```
        "schedule": 21600.0,
        "kwargs": {
            "seed_urls": [
                "https://www.federalregister.gov/documents/current"
            ],
            "allowed_domain": "www.federalregister.gov",
            "max_pages": 50,
            "rate_limit_rps": 0.5,
        },
    },
}
```

Listing 15: Full Celery Beat schedule (app/celery_app.py)

B Glossary

AKN / Akoma Ntoso OASIS open standard for legal documents in XML, used widely by EU and UN bodies.

Beat Celery's built-in periodic task scheduler, analogous to a `cron` daemon.

BFS Breadth-First Search; the traversal algorithm used by the WebConnector to discover and crawl links.

Celery Distributed task queue for Python applications, backed by a message broker (Redis here).

content_hash SHA-256 hexadecimal digest of a document's `raw_text`; the deduplication key.

Crawl4AI Open-source Python library providing an async web crawler backed by Playwright/Chromium.

EUR-Lex Official portal for European Union legal texts.

FCA Financial Conduct Authority; UK financial services and markets regulator.

Flower Real-time Celery monitoring web UI.

feedparser Python library for parsing RSS, Atom, and RDF feeds.

govinfo.gov US Government Publishing Office repository for federal government information.

langdetect Python port of Google's language-detection library; returns ISO 639-1 codes.

lxml High-performance Python XML/HTML processing library using libxml2.

OpenRouter LLM gateway aggregating multiple AI model providers under a single API.

pdfplumber Python library for extracting text and tables from PDF files using pdfminer.six.

Playwright Microsoft's browser automation library; used by Crawl4AI for JS rendering.

RawDocument The core SQLModel/database entity representing one unique ingested document.

spider trap A URL pattern that generates an infinite number of unique URLs, causing a crawler to loop indefinitely.

SQLModel Python library combining SQLAlchemy (ORM) and Pydantic (validation) for type-safe database models.

upsert An `INSERT ...ON CONFLICT DO UPDATE` database operation; inserts if new, updates if exists.

USLM United States Legislative Markup; XML schema used by US Code and Federal Register documents.